Resequencing Blame: An Airport-Game Theory of Cross-Flow Head-of-Line Blocking in Shared Reorder Buffers

Adhyan Jain* Atharva Sheersh Pandey
Kshitij Manikshete
[Institution, department, city, country: to be added]

Abstract

When several flows share one reordering buffer, as in Wi-Fi 7 multi-link operation, a packet can wait for earlier packets of *other* flows. Studies report this head-of-line blocking per flow but do not say which other flow is responsible. We define the attribution game precisely: the players are the other flows, the outcome is the cross-flow blocking of one packet, and a coalition is a set of flows whose packets are kept in a recorded trace with arrival times frozen. For a receiver that releases packets in sequence order this game is an *airport game* (a coalition costs as much as its latest member [9]), so the Shapley value has a closed form computed from observed arrival events, with no counterfactual replays. We prove that leave-one-out blame misses exactly the second-largest lateness, that Shapley blame is L^1-Lipschitz in the lateness with a constant (2) independent of load, number of flows and buffer size, whereas the usual "last arrival takes all" attribution has no bound, and that the structure survives *forced release*: a receiver with a finite window and a hold timer is the reference receiver applied to effective arrival times, with an explicit bound between ideal and forced-release blame and a per-packet applicability certificate that refuses packets whose release was forced. Exact oracles confirm the theory on 500 simulator runs, on 900 traces with windows, timers and permanent loss (249,142 packets, maximum error 5.7e-14), and on the independent reorder buffer of ns-3.47 (3,253,034 packets, release-law error 0). Negative and null results are reported: in the simulator and in ns-3 most blocked packets have a single late flow, so the usual rules coincide there; Shapley blame is not more stable on average under timestamp noise, only in the worst case; one preregistered numerical criterion failed; and the applicability certificate refuses 56% of packets in the stress traces. All evidence is from simulation.

1 Introduction

Wi-Fi 7 multi-link operation (MLO) [8] lets a device transmit over several links at once, so packets of one traffic identifier can arrive out of order and are held in a single reorder buffer until earlier sequence numbers have arrived. The buffer is shared: a packet of flow i can wait for an earlier-numbered packet of flow j that is delayed on a slower link. The MLO literature we found concentrates on traffic allocation, channel-access anomalies and delay models [10, 2]; classical work on *resequencing* studies the delay of a single stream over parallel paths [1, 7]. We found no work that asks the question a designer or operator faces when a latency-sensitive flow is slowed down: *which other flow is responsible, and by how much?*

*Corresponding author; affiliation and email to be added.

The contribution in one sentence. We give a replay-free, per-packet Shapley attribution of cross-flow blocking in a shared reorder buffer. *Players:* the other flows. *Outcome:* the cross-flow blocking $X_k(T)$ of one packet k when only the packets of the flows in T (and of its own flow) are present in a recorded trace, arrival times frozen. *Why no replay is needed:* $X_k(T)$ is the largest "lateness" among the flows in T, an airport game, whose Shapley value is a sorted partial sum computed from the observed arrival events. The same structure holds for receivers with a finite window and a hold timer once arrival times are replaced by effective (resolution) times.

What is and is not new. The release law (a packet leaves when it and all earlier packets have arrived) is classical resequencing; the airport game and its Shapley value are classical cooperative game theory [9, 4]; cooperative games have been applied to queueing costs [12, 13]. To our knowledge the identification of cross-flow reordering blame with the airport game, and the consequences proved here, are new (our search was limited, Section 2); the novelty is the structure and what it gives, not "Shapley applied to delay".

Contributions, ranked by strength of evidence.

1. **Structure** (Theorem 1): cross-flow blocking is an airport game per packet, exactly, for every coalition kept in a trace. Verified by oracles that re-run the *unmodified* receiver on traces with flows deleted (107,080 packets, error $\leq$ 2.3e-13; CONF).

2. **Replay-free exact blame and the degeneracy of the alternatives** (Theorems 2, 3, Example 1): closed-form Shapley blame; leave-one-out misses exactly the second-largest lateness and the releaser rule charges one of several tied flows arbitrarily (ANALYTIC, exact). Cost: $O(m \log m)$ per packet against 2^m replays (22 times faster already at $m = 3$ other flows).

3. **Forced release** (Section 6): a receiver with window W and hold timer τ obeys the same law on effective arrival times (Theorem 5); ideal and forced-release blame differ by at most $2m\delta$ (Corollary 1); a per-packet certificate tells where the reference theory applies (Proposition 2). Confirmed on 900 traces, 18 window/timer/loss configurations, 16,554 forced events (CONF).

4. **A second implementation** (Section 9.3): in ns-3.47's own recipient reorder buffer the release law holds exactly over 3,253,034 packets in 160 scripted three-flow runs, our unmodified receiver reproduces its release times, and ns-3's own cross-wait columns are reproduced from the trace (CONF).

5. **Worst-case stability** (Theorem 4): $\|\phi(\ell') - \phi(\ell)\|_1 \leq 2\|\ell' - \ell\|_1$; the releaser rule has no such bound (worst observed change ratio 1.00 against 348). Not an average-case advantage (CONF bound, EXPL ratio).

6. **Scope of additivity** (Theorem 7): the trace-level cross-blocking game is sub-additive for any arrival process; FIFO M/G/1 waiting is super-additive; in replay the queueing part had $R_Q = 1.89$ and 1.74, intervals above one (CONF).

7. **Bridge to counterfactual externalities and a stochastic baseline** (Sections 7, 8): accounting identity, boundary condition, an explained-fraction bound with violations in 0/13 Python and 0/6 ns-3 held-out cells (CONF); Poisson closed forms serve as a validation baseline, not as a standalone result.

What we do not claim. Shapley is not claimed to be the only reasonable rule (it is characterised under stated axioms); blame-guided scheduling is not claimed to improve performance (an aligned decision test found no benefit); the stochastic model is not claimed to describe any device; the results are not claimed for hardware, for ns-3's own forced-release mechanism, or for duplicates. No human annotation and no model judge were used.

2 Related work and positioning

Searches were limited, not exhaustive; statements of novelty are *to our knowledge*.

- **Resequencing.** Baccelli, Gelenbe and Plateau analyse resequencing delay end to end [1]; Han and Makowski study resequencing delay under multipath routing and its light- and heavy-tailed behaviour [7]. Finite buffers, timeouts and forced release are standard in this literature; we add attribution of a flow's wait to the flows sharing the buffer, and show it survives these mechanisms.
- **Airport games.** Littlechild and Owen give the closed-form Shapley value of the game in which a coalition costs as much as its largest member [9]; Dubey shows that Shapley's axiomatisation remains valid on this class [4]. We use both; our contribution is the identification of resequencing blame with this class.
- **Cooperative games for queues.** Maniquet characterises the Shapley value for queueing problems [12]; Mitra, Gupta and Hemachandra analyse cost sharing in shared multi-class service systems [13]. Those games concern waiting in service queues; none, in our search, concerns the reorder buffer, where the cost of a coalition is a maximum rather than a sum of service interactions.
- **Shapley in networking and learning.** Shapley attribution is used for ISP settlement [11] and for data valuation, where leave-one-out is known to miss interactions [5]; we prove the sharp form of that limitation for this structure (Theorem 3).
- **Queueing externalities.** Naor [14] showed that a customer ignores the delay imposed on others; we use the convexity of FIFO waiting (Theorem 7).
- **Counterfactual replay and testing.** Common random numbers [6], counterfactual trajectories [15] and metamorphic testing [3] motivate the interventional replay and our truncate-and-rebuild oracle.

Reference entries inherited from earlier project notes that we could not verify were left out. We did not compute standard reordering metrics (reorder density, buffer-occupancy density) on our traces.

3 Model

Packets are indexed $k = 1, 2, \ldots$ in the order of their sequence numbers (SN). Packet k belongs to flow $f(k) \in N$ and physically arrives at the receiver at time A_k ($A_k = \infty$ if it never arrives). Flows are an arbitrary partition of the packets that share the buffer (applications, stations, streams).

Definition 1 (Reference receiver)**.** Every sequence number eventually arrives and a packet is released at the first instant at which it and all packets with smaller SN have arrived: no forced advance, no timeout. The release time is R_k. This is an analytically tractable reference, not a model of deployed receivers; Section 6 treats windows and timers.

Lemma 1 (Release law)**.** $R_k = \max_{j \le k} A_j$.

Proof. Packet k can be released once A_j has occurred for every $j \le k$, and release is triggered by the last such arrival. □

Decomposition. For a packet k of flow i let $b_k = \max\{A_j : j \le k,\ f(j) = i\}$ be the time at which flow i's own packets up to k have all arrived. Self-blocking is $S_k = b_k - A_k$ and cross-blocking is $X_k = R_k - b_k \ge 0$, so the reordering wait is $S_k + X_k = R_k - A_k$.

Definition 2 (Lateness)**.** For $j \ne i$ let $M_{j,k} = \max\{A_l : l < k,\ f(l) = j\}$ ($-\infty$ if there is none) and $\ell_{j,k} = (M_{j,k} - b_k)^+$: how long after flow i's own packets are complete the latest earlier packet of flow j arrives.

The game. For $T \subseteq N \setminus \{i\}$ the *restricted trace* keeps the packets of $T \cup \{i\}$ with their arrival times and relative SN order and runs the same receiver; $X_k(T)$ is the cross-blocking of packet k in it. This is a metamorphic operation on a recorded trace that freezes physical arrivals. It differs from an *interventional* replay in which the other flows are removed from the system and the arrivals themselves change (Section 7).

4 Structure: cross-flow blocking is an airport game

Theorem 1 (Airport structure)**.** *For every packet k of flow i and every $T \subseteq N \setminus \{i\}$,*

$$X_k(T) = \max_{j \in T} \ell_{j,k} \qquad (\max \emptyset = 0).$$

In particular $X_k = X_k(N \setminus \{i\}) = \max_{j \ne i} \ell_{j,k}$, and $T \mapsto X_k(T)$ is a monotone, submodular airport game.

Proof. In the restricted trace, the release law gives $R_k(T) = \max\{b_k, \max_{j \in T} M_{j,k}\}$, because the packets with $SN \le k$ are those of i (whose latest arrival is b_k) and of the flows in T. Therefore $X_k(T) = R_k(T) - b_k = \max\{0, \max_{j \in T}(M_{j,k} - b_k)\} = \max_{j \in T} \ell_{j,k}$. A maximum of nonnegative numbers is monotone and submodular in the index set. □

Remark 1. The release time depends on all earlier flows only through their latest arrival. Two or more flows that are both late therefore do not add: the victim waits for the later of them.

5 Blame

Let $m = |N \setminus \{i\}|$ and write $\ell_{(1)} \le \cdots \le \ell_{(m)}$ for the sorted lateness of a packet, $\ell_{(0)} = 0$.

Theorem 2 (Closed-form Shapley blame)**.** *The Shapley value ϕ of the airport game $v(T) = \max_{j \in T} \ell_j$ is, for the flow of rank r,*

$$\phi_{(r)} = \sum_{s=1}^{r} \frac{\ell_{(s)} - \ell_{(s-1)}}{m - s + 1}, \qquad \textit{equivalently} \qquad \phi_j = \int_0^{\ell_j} \frac{dt}{n(t)}, \quad n(t) = \#\{k : \ell_k \ge t\}.$$

It satisfies: (a) efficiency $\sum_j \phi_j = \max_j \ell_j = X_k$; (b) symmetry and the null-player property ($\ell_j = 0 \Rightarrow \phi_j = 0$); (c) rank preservation $\ell_a \le \ell_b \Rightarrow \phi_a \le \phi_b$ and $0 \le \phi_j \le \ell_j$; (d) core membership $\sum_{j \in S} \phi_j \le v(S)$ for every S; (e) additivity over packets, so the blame of j on i over any set of packets is the sum of the per-packet blames. Among rules on airport games satisfying Shapley's axioms it is unique [4]. It is computed in $O(m \log m)$ per packet.

Proof. Let H_s be the set of flows with rank $\geq s$. Then $v = \sum_{s=1}^{m} (\ell_{(s)} - \ell_{(s-1)})\, u_s$ where $u_s(T) = \mathbf{1}[T \cap H_s \neq \emptyset]$ (evaluated at T the right-hand side sums the increments up to the largest rank present in T, i.e. $\ell_{(\max)}$). By symmetry among the members of H_s, the null-player property for the others and efficiency, the Shapley value of u_s gives each member of H_s the amount $1/|H_s| = 1/(m-s+1)$. Linearity of the Shapley value gives the formula; the integral form follows because $n(t) = m - s + 1$ for $t \in (\ell_{(s-1)}, \ell_{(s)}]$. (a)–(c) are read off the formula (every term is nonnegative, $1/(m-s+1) \leq 1$, and $\phi_{(r)} - \phi_{(r-1)} \geq 0$). For (d), $\sum_{j \in S} \phi_j = \sum_s (\ell_{(s)} - \ell_{(s-1)}) |S \cap H_s|/|H_s| \leq \sum_s (\ell_{(s)} - \ell_{(s-1)}) \mathbf{1}[S \cap H_s \neq \emptyset] = v(S)$. (e) is additivity of the Shapley value. Uniqueness is Dubey's theorem [4]; the closed form is the Littlechild–Owen expression [9]. □

Algorithm 1 (trace → blame). Input: packets in SN order with flow $f(k)$ and arrival (or effective arrival) A_k. Maintain, for each flow j, M_j = latest arrival so far.

1. For $k = 1, 2, \ldots$: $b \leftarrow \max(A_k, M_{f(k)})$; for $j \neq f(k)$: $\ell_j \leftarrow (M_j - b)^+$.
2. Sort the ℓ_j; $\phi_{(r)} \leftarrow \sum_{s \leq r} (\ell_{(s)} - \ell_{(s-1)})/(m-s+1)$; add ϕ_j to the blame of j on $f(k)$.
3. $M_{f(k)} \leftarrow \max(M_{f(k)}, A_k)$.

Time $O(Km \log m)$ for K packets and m other flows, memory $O(m)$. Computing the same blame from counterfactual replays needs a run for every coalition (2^m per victim).

Figure 1: Replay-free exact blame.

Theorem 3 (Masking and sub-additivity)**.** *Let* $\mathrm{LOO}_j = v(N') - v(N' \setminus \{j\})$, $N' = N \setminus \{i\}$. *Then* $\mathrm{LOO}_j = \ell_{(m)} - \ell_{(m-1)}$ *if* j *is the unique flow with the largest lateness and* $\mathrm{LOO}_j = 0$ *otherwise. Consequently* $\sum_j \mathrm{LOO}_j \leq X_k \leq \sum_j \ell_j$, *and the amount leave-one-out fails to account for is exactly the second-largest lateness* $\ell_{(m-1)}$.

Proof. Removing a flow changes the maximum only if it was the unique maximiser, and then to the second-largest value. Summing, $\sum_j \mathrm{LOO}_j = \ell_{(m)} - \ell_{(m-1)} \leq \ell_{(m)} = X_k \leq \sum_j \ell_j$; the unaccounted amount is $X_k - \sum_j \mathrm{LOO}_j = \ell_{(m-1)}$; if the maximiser is not unique all $\mathrm{LOO}_j = 0$ and $X_k = \ell_{(m)} = \ell_{(m-1)}$, so the same holds. □

Example 1 (A minimal case where the rules differ)**.** A victim packet waits $X = 3$ ms because two flows are equally late ($\ell = 3, 3$) and a third is a little late ($\ell = 1$). Leave-one-out charges $(0, 0, 0)$: it explains none of the 3 ms. The releaser rule charges $(3, 0, 0)$ or $(0, 3, 0)$, depending on which of the tied packets happened to arrive last. Shapley charges $(\frac{4}{3}, \frac{4}{3}, \frac{1}{3})$: the first millisecond is shared by all three flows that were late at least that long, the remaining two by the two that were late longer. (Checked against subset enumeration.)

Proposition 1 (When the rules coincide)**.** *If at most one other flow has* $\ell_{j,k} > 0$, *then the Shapley, releaser and leave-one-out blames of packet* k *all equal* X_k *on that flow. If two or more flows are late, the releaser and leave-one-out rules charge at most one flow while Shapley charges every late flow* ($\phi_j > 0 \iff \ell_j > 0$).

Theorem 4 (Stability)**.** *Let* $\ell, \ell' \in \mathbb{R}_+^m$ *be lateness vectors of the same packet (for example computed from two versions of the trace). (a) If every arrival timestamp is changed by at most* ϵ, *then* $\|\ell' - \ell\|_\infty \leq 2\epsilon$. *(b)* $\|\phi(\ell') - \phi(\ell)\|_1 \leq 2\|\ell' - \ell\|_1$; *the constant 2 does not depend on* m, *on the load, on the window or on the arrival process, so under (a)* $\|\phi(\ell') - \phi(\ell)\|_1 \leq 4m\epsilon$. *(c) The releaser blame* ρ *has no such bound: for* $\ell = (x, x+\delta)$ *and* $\ell' = (x+\delta, x)$, $\|\rho(\ell') - \rho(\ell)\|_1 = 2(x+\delta)$ *while* $\|\ell' - \ell\|_1 = 2\delta$.

Proof. (a) b_k and each $M_{j,k}$ are maxima of timestamps and move by at most ϵ; the difference moves by at most 2ϵ and $(\cdot)^+$ is 1-Lipschitz. (b) Write $\phi = \int_0^\infty U(t)\,dt$ with $U(t)$ the uniform distribution on $H(t) = \{j : \ell_j \geq t\}$ (zero if empty), the integral form of Theorem 2. If $H'(t) = H(t)$ then $U'(t) = U(t)$; otherwise some j has t between ℓ_j and ℓ'_j, a set of measure at most $\sum_j |\ell'_j - \ell_j|$, on which $\|U'(t) - U(t)\|_1 \leq 2$. (c) is a direct computation. □

The bound is about the lateness-to-blame map; it says nothing about robustness of a deployed measurement chain, and by Theorem 5 it also covers receivers with windows and timers when ℓ is computed from effective arrival times.

6 Forced release: windows and hold timers

A deployed receiver does not wait forever. In the Block-Ack model the reorder buffer holds at most W sequence numbers (we write W for window and buffer capacity); an arriving SN beyond the window forces the window forward, and many implementations also release a packet held longer than a timer τ. A packet that arrives after its slot has been skipped is discarded. We model: (R1) a packet with SN below the window start is discarded; (R2) an arriving SN $s \geq$ window start $+W$ moves the window start to $s - W + 1$ and releases every buffered packet below it in order; (R3) a buffered packet held for τ is released at its arrival time $+\tau$ together with everything buffered below it, the window start moving to it; (R4) after each event, contiguous packets are released. (R1, R2, R4) are the project's `BlockAckReceiver`; (R3) is a reference timer receiver we implement.

Definition 3 (Effective arrival)**.** $F_j = \min\{A_s : s \geq j + W\}$, $T_j = \min\{A_s : s > j\} + \tau$ ($T_j = \infty$ without a timer), and $\tilde{A}_j = \min(A_j, F_j, T_j)$: the time at which slot j is *resolved*, by its own arrival or by a forced advance.

Theorem 5 (Release law with window and timer)**.** *Every released packet k has release time $R_k = \max_{j \leq k} \tilde{A}_j$, and the delivered set is $\{k : A_k < \min(F_k, T_k)$ and $R_k < \infty\}$.*

Proof. The window start moves only by (i) contiguous release, which needs the slot resolved, (ii) a forced advance, triggered by the arrival of an SN $s \geq j + W$ (the earliest such arrival is F_j; while slot j is unresolved the window start is at most j, so any $s \geq j + W$ triggers it), or (iii) a timer expiry of a buffered $s > j$ (the earliest is T_j; while j is unresolved every arrived $s > j$ stays buffered, so its timer runs). Hence slot j is resolved at the first of A_j, F_j, T_j. If A_j is first, packet j fills its slot; otherwise the slot is skipped and packet j, if it ever arrives, is discarded by (R1). Packet k is released when all slots $\leq k$ are resolved, i.e. at $\max_{j \leq k} \tilde{A}_j$, and it is delivered iff its own slot was filled by it. □

Theorem 6 (Airport structure on effective arrivals)**.** *With $\tilde{b}_k$, $\tilde{\ell}_{j,k}$ defined as $b_k, \ell_{j,k}$ with $\tilde{A}$ in place of A (a skipped slot belongs to the flow of the packet that should have filled it), the cross-blocking of a released packet $\tilde{X}_k = R_k - \tilde{b}_k$ equals $\max_{j \neq f(k)} \tilde{\ell}_{j,k}$, and Theorems 2–4 hold verbatim for $\tilde{\ell}$.*

Proof. By Theorem 5 the receiver releases exactly as the reference receiver would if packet j arrived at $\tilde{A}_j$; apply Theorems 1–4. □

Corollary 1 (Bounded error between ideal and forced-release blame)**.** *On a loss-free trace let $\delta_k = \max_{j \leq k}(A_j - \tilde{A}_j) \geq 0$. Then $|\tilde{\ell}_{j,k} - \ell_{j,k}| \leq \delta_k$ for every j, hence $\|\phi(\tilde{\ell}) - \phi(\ell)\|_1 \leq 2m\delta_k$; blame is identical when no slot up to k was resolved early ($\delta_k = 0$).*

Proof. $0 \leq A_l - \tilde{A}_l \leq \delta_k$ for $l \leq k$, so $\tilde{b}_k - b_k$ and $\tilde{M}_{j,k} - M_{j,k}$ both lie in $[-\delta_k, 0]$ and their difference in $[-\delta_k, \delta_k]$; $(\cdot)^+$ is 1-Lipschitz. Then Theorem 4(b) and $\|\cdot\|_1 \leq m\|\cdot\|_\infty$. □

Proposition 2 (Applicability certificate)**.** *Call a released packet k* certified *if $R_k = \max\{A_j : j \le k,\ j \text{ received}\}$. For a certified packet the identity $X_k = \max_j \ell_{j,k}$ computed from observed arrivals holds exactly (it is the identity itself). If no slot $\le k$ was resolved before its physical arrival ($\tilde{A}_j = A_j$ for all $j \le k$) the packet is certified; equivalently, a rejection implies that a forced release or a loss preceded k (soundness). The converse fails: a packet can be certified although an earlier slot was skipped, when the cut did not bind.*

Remark 2 (What is and is not covered)**.** Theorem 5 is for the arrival-triggered window advance of (R2) and the timer (R3). In ns-3 the originator moves the recipient window after a drop by a different trigger (a block-ack request); that mechanism, duplicates, and real hardware are not covered. The skipped slot's owner is known at the sender but not at the receiver, so charging a forced release to the flow that caused the gap needs sender-side information; a receiver-only trace can attribute only certified packets.

7 From traces to interventions

The airport structure is a statement about a *frozen* trace. The counterfactual an operator cares about, "how much worse is flow i's delay because flow j exists?", lets arrivals change: removing a flow also changes queueing and contention. With $D_i(C)$ the mean end-to-end delay of i when exactly the flows $C \ni i$ are present (identical arrivals and per-packet noise for a flow in every coalition, by keyed random numbers [6]), the *interventional externality* is $E_i = D_i(N) - D_i(\{i\})$. Three results connect the two levels (proved and tested in the companion measurement study; restated with proofs here):

Proposition 3 (Accounting identity)**.** $E_i = \Delta\mathrm{Net}_i + \Delta S_i + \Delta X_i$, *with* $\Delta Z_i = Z_i(N) - Z_i(\{i\})$, *and* $\Delta X_i = X_i(N)$ *is exactly the trace-level cross-blocking.*

Proof. Per packet e2e = Net + S + X; average and subtract; a single flow has $X = 0$. □

Proposition 4 (Boundary condition)**.** *If adding or removing flow j changes neither the arrival times nor the sequence numbers of flow i's packets (e.g. flows pinned to disjoint links), then* $\Delta\mathrm{Net}_i = \Delta S_i = 0$ *and* $E_i = X_i(N)$.

Proposition 5 (Explained-fraction bound)**.** *If* $E_i > 0$ *and* $d_{\min} \le D_i(\{i\})$, *then* $r := X_i(N)/E_i \ge X_i(N)/(D_i(N) - d_{\min}) \ge q := X_i(N)/D_i(N)$, *and q is computable from one trace.*

Proof. $E_i = D_i(N) - D_i(\{i\}) \le D_i(N) - d_{\min}$ and $X_i(N) \ge 0$. □

Theorem 7 (Scope of additivity)**.** *Three set functions, three scopes. (i)* Trace level, any arrival process: *the cross-blocking game $v_k(T) = X_k(T)$ satisfies $\sum_j \mathrm{LOO}_j \le X_k \le \sum_j X_k(\{j\})$. (ii)* Static FIFO M/G/1 only: *the mean waiting time $W(T)$ of a victim sharing a work-conserving FIFO single server with the Poisson flows in T is supermodular in T, hence $\sum_j [W(N) - W(N \setminus \{j\})] \ge W(N) - W(\emptyset)$. (iii)* Interventional replay, algebra: *with $E = E_Q + X$, $E_Q = \Delta\mathrm{Net} + \Delta S$, and $R_Z = \sum_{i,j} \mathrm{LOO}^Z_{j\to i} / \sum_i \Delta Z_i$,*

$$R - 1 = \frac{(R_Q - 1)\, E_Q + (R_X - 1)\, X}{E}.$$

Proof. (i) is Theorem 3. (ii) Write $W(T) = a(T)h(\rho(T))$ with $a(T) = \sum_{j\in T} \lambda_j E[S_j^2]$, $\rho(T) = \sum_{j\in T} \lambda_j E[S_j] < 1$ and $h(\rho) = 1/(2(1-\rho))$ convex and increasing (Pollaczek–Khinchine). The marginal $W(T \cup \{j\}) - W(T) = a_j h(\rho_T + \rho_j) + a(T)[h(\rho_T + \rho_j) - h(\rho_T)]$ is a sum of nonnegative

terms that are nondecreasing in T; so W is supermodular. For supermodular W with $W(\emptyset) = 0$, $W(N) = \sum_j [W(S_j \cup \{j\}) - W(S_j)] \le \sum_j [W(N) - W(N \setminus \{j\})]$ along any ordering S_j of predecessors. (iii) Leave-one-out and totals are additive across the components of E. □

Remark 3. We make no universal claim about Wi-Fi channel access: (ii) is a stylised single-server statement, and for the simulator the super-additivity of the queueing part is tested, not assumed (Section 9.4).

8 A stochastic baseline: Poisson flows

Stochastic resequencing models are standard [1, 7]; we include a closed form for the blocking game only as an exact baseline against which the trace-level machinery and the simulator can be checked, not as a standalone result. Consider flows j that are Poisson with rate λ_j, each packet taking an independent path delay d with law F_j ("infinite-server" paths), sequence numbers assigned in send order. Let $\mu_j(x) = E[(d_j - x)^+]$ and $\Lambda_j(x) = \lambda_j \mu_j(x)$, and consider a tagged packet of flow i sent at time 0 with delay $d_0 \sim F_i$.

Theorem 8 (Expected blocking and blame)**.** *(a) The expected cross-blocking when the coalition T of other flows is present is*

$$v(T) = \int_0^\infty F_i(x)\, e^{-\Lambda_i(x)} \Big(1 - e^{-\sum_{j \in T} \Lambda_j(x)}\Big) dx.$$

(b) The expected Shapley blame of flow j is

$$\bar{\phi}_j = \int_0^\infty F_i(x) e^{-\Lambda_i(x)} \big(1 - e^{-\Lambda_j(x)}\big) \int_0^1 \prod_{k \neq i,j} \big(1 - t\,(1 - e^{-\Lambda_k(x)})\big)\, dt\, dx,$$

and $\sum_j \bar{\phi}_j = v(N \setminus \{i\})$.

Proof. Packets of flow j sent before time 0 and their arrival times form, by the displacement theorem for Poisson processes, an independent Poisson process; the expected number arriving after $x \ge 0$ is $\lambda_j \int_0^\infty P(d_j > x + s) ds = \Lambda_j(x)$, so $P(M_j \le x) = e^{-\Lambda_j(x)}$ and the M_j are independent. The own-flow completion time is $b = \max(d_0, M_i)$ with $P(b \le x) = F_i(x) e^{-\Lambda_i(x)}$, independent of $M_T = \max_{j \in T} M_j$. By Theorem 1, $X(T) = (M_T - b)^+ = \int_0^\infty \mathbf{1}[b \le x < M_T] dx$, and taking expectations gives (a). For (b), the Shapley value is linear in the game, so the expected airport-Shapley blame is the Shapley value of v. For each x the game $T \mapsto 1 - \prod_{j \in T} q_j$, $q_j = e^{-\Lambda_j(x)}$, has multilinear extension $W(p) = 1 - \prod_k (1 - p_k(1 - q_k))$, and Owen's formula [16] gives $\phi_j = \int_0^1 \partial_j W(t, \dots, t)\, dt$. □

9 Validation

Design. Studies 5 and 6 were preregistered (hypotheses, thresholds, seeds, hashed and committed before any confirmatory run; exploratory analyses declared in hashed amendments before the run; the Study 6 amendment adding timers, the bounded-error comparison and the certificate was written before hashing, in response to an adversarial review). Runners refuse to execute if a hash has changed or results exist. Intervals are bootstrap percentile intervals over seeds. "Oracles" are label-free exact identities.

9.1 Reference-receiver oracles on the project simulator (H16)

O1 release time equals the prefix maximum; O2 cross-blocking equals the maximum lateness; O3 closed-form Shapley equals subset-enumeration Shapley and sums to X_k; O4 *truncate and rebuild*: delete flows from the trace, run the unmodified `BlockAckReceiver`, compare with $\max_{j \in T} \ell_{j,k}$; O5 the releaser blame from lateness equals the simulator's own attribution; O6 $\sum \mathrm{LOO} \le X \le \sum \ell$ and the formula of Theorem 3; O7 core membership. Data: 500 simulator runs (S4 and S8; FIFO and FullSASP; two operating points) and 200 runs of two non-Wi-Fi generators (G2: five streams multiplexed on one ordered connection with loss and retransmission delay; G3: heavy-tailed Pareto paths), 300 packets each. All oracles hold (maximum error 2.3e-13, 0 violations), including the

cell	runs	packets	blocked	one late flow	forced adv.	max error
S4 FIFO (1.0, 0.1)	100	23,695	1.5%	97%	0	2.2e-16
S4 FullSASP (1.0, 0.1)	100	23,693	1.3%	95%	0	2.2e-16
S4 FIFO (0.75, 3.0)	100	17,629	27.5%	100%	0	8.9e-16
S8 FIFO (1.0, 0.1)	100	23,655	1.9%	100%	0	1.8e-15
S8 FIFO (1.0, 3.0)	100	21,803	38.5%	100%	0	9.8e-15
G2 multiplexed streams	100	30,000	52.0%	68%	0	1.4e-14
G3 heavy-tailed paths	100	30,000	52.6%	71%	0	2.3e-13

Table 1: H16 (CONF): oracle errors over all runs. "one late flow" is the share of blocked packets with exactly one late flow. Maximum error over O1–O6; tolerance 10^{-9}; O6/O7 violation counts 0. Cells are (load multiplier, link-2 extra delay in ms).

generators, and no run had a forced window advance. In the simulator cells 95%–100% of blocked packets have exactly one late flow, so by Proposition 1 the three blame rules coincide there; in G2 and G3 only 68%–71% do.

9.2 Forced release: windows, timers and loss (H20, H21)

Generator G4 (Poisson sends, heavy-tailed delays, permanent loss with probability 0.05 or 0, window $W \in \{8, 16, 64\}$, hold timer $\tau \in \{\text{none}, 3, 10\}$ ms) was passed through the receiver of Section 6: 900 traces, 249,142 packets, 16,554 forced events, 96.8% of traces with at least one. **H20 confirmed:** the release law of Theorem 5 has error 0 for every released packet and the delivered set has 0 mismatches. **H21 confirmed:** on effective arrivals the oracles of Section 9.1 hold (maximum error 5.7e-14, 0 violations); on the loss-free configurations the bounds of Corollary 1 have 0 violations (the largest observed blame change is 0.18 of the bound); the certificate has 0 false rejections. The certificate is conservative in the other direction: only 44% of delivered packets are certified in these stress traces (82,184 packets are certified although an earlier slot was skipped), and 13%–90% of blocked packets owe their largest lateness to a skipped slot, which is why the reference theory alone is not enough once forced releases occur. The timer receiver is our reference implementation, so the timer part tests the law against our reading of (R3), not against a third-party timer.

9.3 A second implementation: ns-3's reorder buffer (H22–H24)

The frozen ns-3.47 experiment (patched with observer hooks that record reorder-buffer events without changing its logic) exposes the per-packet insert and release times of ns-3's own recipient Block-Ack reorder buffer, third-party code. We ran 160 scripted three-flow scenarios (two asymmetry conditions, two load scales, 40 seeds, FIFO) with Poisson traffic. **H22 confirmed:** for all 3,253,034 packets checked (873,950 blocked) the release law of Lemma 1 holds with error 0 ms

$(W, \tau, p_{\text{loss}})$	traces	forced events/trace	delivered	blocked from skipped slot	certified
(8, none, 0.05)	50	24.6	91%	57%	38%
(8, none, 0.0)	50	12.3	96%	35%	45%
(8, 3, 0.05)	50	34.4	87%	60%	34%
(8, 3, 0.0)	50	23.8	91%	47%	40%
(8, 10, 0.05)	50	25.7	91%	57%	37%
(8, 10, 0.0)	50	10.9	96%	31%	53%
(16, none, 0.05)	50	17.9	92%	67%	35%
(16, none, 0.0)	50	4.2	99%	24%	64%
(16, 3, 0.05)	50	32.2	87%	58%	37%
(16, 3, 0.0)	50	23.7	91%	47%	39%
(16, 10, 0.05)	50	20.1	93%	61%	40%
(16, 10, 0.0)	50	6.5	98%	30%	55%
(64, none, 0.05)	50	11.9	81%	90%	15%
(64, none, 0.0)	50	0.5	100%	13%	82%
(64, 3, 0.05)	50	33.9	87%	62%	32%
(64, 3, 0.0)	50	22.6	92%	46%	40%
(64, 10, 0.05)	50	19.3	93%	62%	37%
(64, 10, 0.0)	50	6.6	98%	29%	59%

Table 2: H20/H21 (CONF): 18 receiver configurations, 50 traces each. Forced events are window advances plus timer expiries; "blocked from skipped slot" is the share of blocked packets whose largest lateness comes from a slot that was resolved early; "certified" is the share of delivered packets certified by Proposition 2.

and 0 violations; no run had a forced advance (0 excluded). **H23 confirmed:** our unmodified `BlockAckReceiver`, fed with ns-3's arrival times, reproduces ns-3's release times with error 0 ms (0 mismatches), and the airport oracles hold on these arrivals (maximum error 0). **H24 confirmed:** the trace reproduces the binary's own cross- and within-wait columns to 4.8e-10 and 5.0e-11 ms, and the maximum buffer occupancy reconstructed from the trace equals ns-3's own in 160/160 runs. Among blocked packets 1.8%–16.4% have two or more late flows (the high end at the higher load): most blocked packets in ns-3 also have a single late flow. What this does and does not show: the receiver semantics are independent of ours, while the traffic, scenarios and metrics are ours, and ns-3's own forced-release mechanism (block-ack-request driven) never occurred and is not covered.

A definition gap we found and measured (D2). ns-3's `cross_wait_ms` charges the *whole* wait of a packet to a cross-flow binding blocker, whereas the pure cross-blocking X excludes the self part. The companion studies used the ns-3 column as "X". On the main-campaign configuration for conditions A1 and A2 at loads L0–L2 (two flows, 60 runs, FIFO; condition A0 was not measured) the pure X is 99.5% of the ns-3 quantity overall and at least 98.8% in every cell (self-blocking is 0.012 of X), so for these conditions the earlier ns-3 "X" overstates the pure X by about one percent at most.

9.4 Sub-additive reordering, super-additive queueing (H17)

All 15 coalitions of the S4 system were replayed (keyed random numbers), 100 seeds in each of two cells. H17a (preregistered): the interval of R_Q lies above one in both cells; **confirmed**. The replay cross-blocking part has $R_X = 0.77$ [0.60, 0.94] and 0.98 [0.92, 1.03] (the second interval includes one), consistent with Theorem 7(i) but not an additional confirmation, because that statement is exact only for frozen traces. As exploratory X9, the closed-form blame of the full-run trace differs from the replay Shapley of the cross-blocking component by a relative L^1 error of 0.14 [0.11,

cell	$\bar{X}$ (ms)	$\bar{S}$ (ms)	ns-3 cross wait (ms)	$\bar{X}$ / ns-3 cross
A1 / L0	0.018	0.000	0.018	100.0%
A1 / L1	0.029	0.000	0.029	100.0%
A1 / L2	0.062	0.001	0.063	98.8%
A2 / L0	0.117	0.000	0.117	100.0%
A2 / L1	0.163	0.000	0.163	100.0%
A2 / L2	0.241	0.006	0.244	99.0%

Table 3: D2 (descriptive): pure cross-blocking X, self-blocking S and ns-3's cross wait for the latency-sensitive flow, main-campaign configuration, 10 seeds per cell.

cell	cross share of E	R_Q	R_X (replay)	R (total)
S4 FIFO (1.0, 0.1)	0.6%	1.89 [1.85, 1.93]	0.77 [0.60, 0.94]	1.88
S4 FIFO (0.75, 3.0)	15.4%	1.74 [1.67, 1.80]	0.98 [0.92, 1.03]	1.62

Table 4: H17 (CONF for R_Q; R_X is EXPL): non-additivity ratios of Theorem 7(iii), interventional replay, seed-cluster intervals. The coalition identity gap is at most 2.7e-15 and replay-signature violations are 0.

0.29] and 0.04 [0.03, 0.09]; against the replay Shapley of *total* delay it is 0.99 and 0.85, because cross-blocking is only 0.6% and 15.4% of the externality in these cells.

9.5 Stability (H18, X12, X13)

Independent uniform noise of half-width $\epsilon \in \{0.01, 0.1, 1\}$ ms was added to every arrival timestamp and the blame recomputed. H18a (exact): zero violations of Theorem 4(a),(b) in all conditions (0);

cell, ϵ (ms)	packets	mean change Shapley	mean change releaser	max ratio Shapley	max ratio releaser
C1, 0.1	194	0.057	0.057	1.00	1.0
C2, 0.1	165	0.054	0.054	1.00	1.0
C3, 0.1	2407	0.067	0.067	1.00	1.0
C5, 0.01	4264	0.007	0.007	1.00	1.0
C5, 0.1	4264	0.065	0.065	1.00	1.0
C5, 1.0	4264	0.610	0.610	1.00	1.0
G2, 0.01	15595	0.008	0.007	1.00	1.0
G2, 0.1	15595	0.076	0.067	1.00	18.1
G2, 1.0	15595	0.723	0.902	1.00	347.8
G3, 0.01	15784	0.008	0.007	1.00	208.4
G3, 0.1	15784	0.075	0.070	1.00	82.7
G3, 1.0	15784	0.729	0.679	1.00	40.0

Table 5: Stability (H18 cells C1–C3 at $\epsilon = 0.1$ ms and exploratory X12 cells C5, G2, G3). Mean L^1 change of the per-packet blame vector over blocked packets; maximum over packets of (change of blame)/(L^1 change of lateness).

confirmed. H18b (preregistered: releaser blame changes more than Shapley blame on average) is **not confirmed**: in 12 of 12 simulator cell-levels the two changes are identical (one late flow), and on the generators the releaser change is smaller at small ϵ (at $\epsilon = 0.1$ ms: Shapley 0.076, releaser 0.067 in G2; the sign flips at $\epsilon = 1$ ms with an interval including zero). The valid statement is the worst case: the ratio of blame change to lateness change never exceeded 1.00 for Shapley (bound

2) but reached 348 for the releaser rule (X13, EXPL).

9.6 Stochastic baseline (H19, X15)

Eight parameter sets (one to four competing flows; link mixtures, deterministic and heavy-tailed delays) were simulated with 10^5 draws each. The closed forms of Theorem 8 agreed with Monte Carlo within 2.69 standard errors over 66 quantities and the Shapley sums equalled $v(N \setminus \{i\})$ to 9.2e-10 (relative). **H19 is nevertheless not confirmed as preregistered:** the numerical-convergence criterion (doubling the quadrature grid changes every value by less than 10^{-6}) was exceeded, by up to 7.0e-4, in two sets, because of the quadrature of non-smooth integrands (a deterministic delay gives a step distribution function). A post hoc refinement (POST HOC, `posthoc_h19_refine.py`) shows the heavy-tailed set converging to 4.3e-7 (maximum $|z|$ unchanged at 2.69) and the deterministic-delay set converging slowly (2.6e-4 between the two finest grids, $|z| \leq 1.21$): we attribute the failure to numerical integration, not to the theorem. As a predictor

cell (load, $\Delta\mu$)	observed $\bar{X}$ (ms)	predicted / observed	interval	blame L1 error
(0.1, 2.0)	0.012	1.19	[0.61, 3.37]	0.81
(0.25, 2.0)	0.049	1.33	[1.16, 1.57]	0.37
(0.5, 3.0)	0.285	1.23	[1.16, 1.29]	0.23

Table 6: X15 (EXPL): a-priori prediction in the Python simulator (S4 FIFO) from flow rates and the empirical delay law of calibration seeds, against observed cross-blocking on held-out seeds; blame error is the relative L^1 error of the 12-entry blame matrix.

(X15, exploratory) the model, which assumes independent delays and send-order sequence numbers, overestimates mean cross-blocking by about 20–33% (1.19, 1.33, 1.23 in three cells; the lowest-load interval is wide) with a blame-matrix relative L^1 error of 0.23–0.81: an order-of-magnitude predictor, not a calibrated model.

9.7 Bridge results from the companion studies

For the counterfactual level the companion studies give (all simulation): pinned disjoint links make trace and intervention agree (gap 0.1% against 99.5% for shared links; one preregistered exact check failed with a single-row diagnosis); in ns-3 the cross-blocking share of the externality ranges from 4% to 88% across nine conditions, so it is regime dependent; the explained-fraction bound (Proposition 5) had violations in 0/13 held-out Python cells and in 0/6 ns-3 cells (at the point estimate), and q ranked the cells with Spearman $\rho = 0.93$ ([0.78, 0.98]; within platform 0.85 and 0.60); an aligned decision test found no advantage of Shapley over trace-level blame for choosing a flow to remove (EXPL).

10 Scope, assumptions and limitations

What the theory covers. The reference receiver (Definition 1); receivers with an arrival-triggered window advance and a hold timer (Theorem 5); frozen-trace coalitions. It does not cover ns-3's block-ack-request-driven release, duplicates (not injected in any stress trace), reordering metrics other than blame, or any hardware behaviour.

Timestamps. Lateness uses arrival times at the reorder buffer, available on the receiving device but not from the air; Theorem 4 bounds the lateness-to-blame map, not a measurement chain. Charging a forced release to the owner of a never-received slot needs sender-side information (Section 6).

Why Shapley. Dubey's characterisation makes Shapley the unique rule with efficiency, symmetry, null-player and additivity on this class [4]; the releaser rule is not continuous (Theorem 4(c)) and charges one of several tied flows (Example 1). Other allocations (nucleolus, serial rules) answer other questions; we did not compare them.

Depth. The release law is classical and the structure theorem is short. The contribution is the identification and its consequences (closed form, degeneracy of the alternatives, forced-release extension with a bound and a certificate, stability), each proved and tested; reviewers may rate the technical depth as modest.

Evidence base. One purpose-built simulator, synthetic generators, and ns-3.47's reorder buffer with our scenarios; no hardware, no public external trace, no reordering-metric comparison (reorder density, buffer-occupancy density). The ns-3 forced-release regime was never exercised, and the timer receiver is ours. In the simulator and in ns-3 most blocked packets have a single late flow, so the practical difference between blame rules is small there; it matters in multi-blocker regimes (the generators; the high-load ns-3 cells). We make no claim that blame-guided scheduling improves delay.

Failures and null results kept in the paper. The Study 2 validity control failed; one preregistered exact check (V7) failed with a single-row diagnosis; one preregistered design (H12) was flawed; H18b and H19 were not confirmed as preregistered; the certificate is conservative.

11 Conclusion

A receiver trace tells us what a packet waited for; the airport structure tells us exactly how that wait is composed across flows, and the composition survives the forced releases of windows and timers once arrivals are replaced by resolution times. The result is an exact, replay-free Shapley blame with a closed form, a precise statement of why leave-one-out and the releaser rule are degenerate, an explicit bound and certificate for forced release, a worst-case stability bound, and independent confirmation on ns-3's reorder buffer. The claims are bounded by what failed or was not done: the stability advantage is worst-case only, most blocked packets in the studied systems have a single late flow, the certificate is conservative, and hardware, duplicates and ns-3's own forced release remain untested.

Reproducibility

All materials are in `validation/pairing/`. Study 5: preregistration commit `aba3a8e` (SHA-256 `3fff5423...`), exploratory plan and runner `d8ebc33` (`01390d8a...`), results `c2a745d`. Study 6: preregistration, library, runner and tests `817ad6e` (`8ccbee59...`), results `0bf58a1`. Raw-file hashes: `results/STUDY5_RAW.sha256`, `results/STUDY6_RAW.sha256`. Libraries `airport.py` (frozen at the Study 5 preregistration commit), `airport_fast.py`, `study6.py`; tests `tests/test_study5.py`,

`tests/test_study6.py`. ns-3 per-packet traces are regenerated deterministically from the stored seeds and scripts. Companion studies and hashes are listed in the measurement paper. Numbers: `paper/theory/make_numbers_theory.py`.

Code and data availability

All code, preregistrations, raw result files, hashes and the decision log are in the project repository under `validation/pairing/`; start with `README.md`, which lists every study, command and hash file. Simulator source is `src/mlo_sim/`; the ns-3.47 experiment and its patches are in `validation/ns3/`. A public archive link will be added at submission: [URL to be added].

Declarations

Funding and competing interests: [to be completed by the authors]. Use of AI assistance: an AI coding assistant (Claude Code) was used to write and run code, draft text and perform the audits described in the repository; the authors are responsible for the content. No human annotation and no model-based judge were used in any evaluation.

References

[1] François Baccelli, Erol Gelenbe, and Brigitte Plateau. An end-to-end approach to the resequencing problem. *Journal of the ACM*, 31(3):474–485, 1984.

[2] Marc Carrascosa-Zamacois, Giovanni Geraci, Lorenzo Galati-Giordano, Anders Jonsson, and Boris Bellalta. Understanding multi-link operation in Wi-Fi 7: Performance, anomalies, and solutions. In *IEEE PIMRC*, 2023. arXiv:2210.07695.

[3] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018.

[4] Pradeep Dubey. The Shapley value as aircraft landing fees—revisited. *Management Science*, 28(8):869–874, 1982.

[5] Amirata Ghorbani and James Zou. Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, pages 2242–2251, 2019.

[6] Paul Glasserman and David D. Yao. Some guidelines and guarantees for common random numbers. *Management Science*, 38(6):884–908, 1992.

[7] Yijie Han and Armand M. Makowski. Resequencing delays under multipath routing — asymptotics in a simple queueing model. Technical Report TR 2005-96, Institute for Systems Research, University of Maryland, 2005.

[8] IEEE. IEEE std 802.11be-2024, amendment 8: Enhancements for extremely high throughput (EHT), 2024.

[9] S. C. Littlechild and G. Owen. A simple expression for the Shapley value in a special case. *Management Science*, 20(3):370–372, 1973.

[10] Álvaro López-Raventós and Boris Bellalta. Multi-link operation in IEEE 802.11be WLANs. *IEEE Wireless Communications*, 29(4):94–100, 2022.

[11] Richard T. B. Ma, Dah Ming Chiu, John C. S. Lui, Vishal Misra, and Dan Rubenstein. Internet economics: The use of Shapley value for ISP settlement. *IEEE/ACM Transactions on Networking*, 18(3), 2010.

[12] François Maniquet. A characterization of the Shapley value in queueing problems. *Journal of Economic Theory*, 109(1):90–103, 2003.

[13] Anirban Mitra, Manu K. Gupta, and N. Hemachandra. Cooperative game theoretic analysis of shared services. In *Performance Evaluation Methodologies and Tools (VALUETOOLS 2022)*. Springer, 2023.

[14] P. Naor. The regulation of queue size by levying tolls. *Econometrica*, 37(1):15–24, 1969.

[15] Michael Oberst and David Sontag. Counterfactual off-policy evaluation with Gumbel-Max structural causal models. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *PMLR*, pages 4881–4890, 2019.

[16] Guillermo Owen. Multilinear extensions of games. *Management Science*, 18:64–79, 1972.